Neural Networks as Continuous Dynamical Systems: A Study Using Differential Equations

Somesh Biswas
Roll Number: BSD-CC-2420
Email: someshbiswas71@gmail.com
Indian Statistical Institute, Kolkata
Mathematics IV Project

April 14, 2026

Abstract

Reading through Chen et al.'s work on Neural Ordinary Differential Equations, one observation stood out more than any other: a standard Residual Network layer is mathematically identical to a single forward Euler step. This project takes that observation seriously and builds on it. By treating network depth as a continuous variable, the forward pass of a ResNet can be recast as an Initial Value Problem governed by an ODE — and from there, the tools of classical analysis become directly applicable. This report examines what that shift actually buys us: we apply the Picard-Lindelöf Theorem to settle existence and uniqueness of the network's output, verify Lipschitz continuity for common activation functions like Tanh and ReLU, and reformulate gradient-based training as a Boundary Value Problem via the Adjoint Sensitivity Method. We also look at where continuous models fundamentally fall short — specifically, the topological constraint that prevents ODE-based networks from learning certain simple mappings in low dimensions. Computational experiments comparing discrete and continuous forward passes are included throughout.

Contents

1 Introduction

The starting point for this project was a single equation in Chen et al.'s Neural ODE paper. Written out, a ResNet update looks like this:

$$h_{t+1} = h_t + f(h_t, \theta_t) \tag{1}$$

A forward Euler step looks like this:

$$h_{n+1} = h_n + \Delta t \cdot f(h_n, t_n) \tag{2}$$

Set $\Delta t = 1$ and they are the same thing. That equivalence is not just a notational coincidence — it means that every time someone stacks residual blocks, they are implicitly running a numerical integrator on an underlying continuous system, just with a very coarse step size.

Once that connection is made, a natural question follows: what happens if we take it seriously? Instead of treating network depth as a discrete index, we treat it as a continuous variable $t \in [0, T]$ and define the hidden state's evolution through an ODE. The forward pass becomes an Initial Value Problem. Training becomes, somewhat surprisingly, a Boundary Value Problem.

This project traces that path using the tools from the Mathematics IV syllabus. The Picard-Lindelöf Theorem tells us when the network's output is guaranteed to be unique and well-defined. Lipschitz analysis tells us which activation functions are safe to use. The adjoint method gives us a way to compute gradients without storing the entire forward trajectory. And a topological argument shows where these continuous models hit a hard wall — things they simply cannot learn, not due to insufficient parameters, but due to the geometry of ODE solutions.

2 Mathematical Preliminaries

To rigorously study continuous networks, we must first establish the foundational mathematical definitions that bridge numerical analysis and deep learning.

2.1 Initial Value Problems (IVPs)

An Ordinary Differential Equation describes how a variable evolves over continuous time. An Initial Value Problem (IVP) pairs an ODE with a specified starting state.

Definition 1 (Initial Value Problem)**.** *Given an open domain $D \subset \mathbb{R} \times \mathbb{R}^n$ and a continuous function $f : D \to \mathbb{R}^n$, an Initial Value Problem is formulated as:*

$$\frac{dh(t)}{dt} = f(h(t), t), \quad h(t_0) = h_0 \tag{3}$$

where $h(t)$ is the unknown state vector, and h_0 is the known initial condition at time t_0.

2.2 Euler's Method for Numerical Integration

Since most non-linear ODEs cannot be solved analytically, numerical integration techniques are employed. The simplest of these is the forward Euler method. Given a step size Δt, the continuous time domain is discretized into steps $t_n = t_0 + n\Delta t$. The derivative is approximated using finite differences:

$$\frac{h(t_{n+1}) - h(t_n)}{\Delta t} \approx f(h(t_n), t_n) \tag{4}$$

Rearranging this yields the Euler update rule:

$$h_{n+1} = h_n + \Delta t \cdot f(h_n, t_n) \tag{5}$$

3 From Discrete Networks to Continuous Dynamics

3.1 Residual Networks (ResNets)

In a standard Multi-Layer Perceptron (MLP), a hidden layer transforms data via $h_{t+1} = f(h_t, \theta_t)$. However, in Deep Residual Networks (ResNets)—which revolutionized computer vision by allowing for incredibly deep architectures—the network learns a residual mapping. A skip-connection is added so the input bypasses the transformation and is added to the output:

$$h_{t+1} = h_t + f(h_t, \theta_t) \tag{6}$$

where f is a non-linear neural network block parameterized by weights θ_t.

3.2 The Continuous Limit

Placing Equation 6 and Equation 5 side by side makes something immediately obvious: a ResNet update is just a forward Euler step with $\Delta t = 1$. The non-linear block $f(h_t, \theta_t)$ plays the role of the derivative, and the skip connection is exactly the h_n carry-over term.

If we treat the layer index not as a discrete counter but as a continuous variable $t \in [0, T]$ representing network depth, and let $\Delta t \to 0$, the discrete update converges to a continuous first-order ordinary differential equation:

$$\frac{dh(t)}{dt} = f(h(t), \theta(t), t) \tag{7}$$

In this continuous framework, passing an input x through the network is identical to solving an IVP with initial condition $h(0) = x$, integrated from $t = 0$ to an output horizon $t = T$.

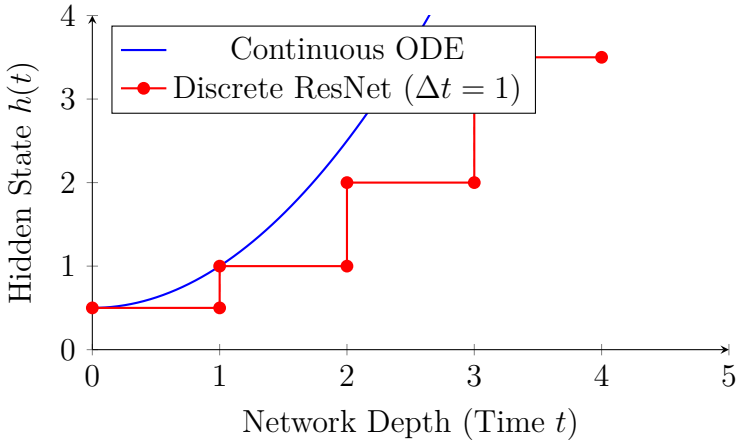

Figure 1: Comparison of a discrete ResNet (modeled as Euler steps) versus the smooth trajectory of a continuous Neural ODE.

4 Existence and Uniqueness: Picard's Theorem

When utilizing continuous dynamics for machine learning, a critical theoretical question arises: given an input x, is the output of the network guaranteed to be deterministic and well-defined? If multiple solutions exist for the ODE, the network would map a single input to multiple different outputs, rendering it useless for prediction tasks.

To guarantee well-posedness, we apply the Picard-Lindelöf Theorem.

Theorem 1 (Picard-Lindelöf)**.** *Consider the IVP* $\frac{dh}{dt} = f(h,t)$ *with* $h(t_0) = h_0$*. If* f *is continuous in* t *and uniformly Lipschitz continuous in* h *over a region* D *containing* (t_0, h_0)*, then there exists a unique solution* $h(t)$ *to the IVP on some interval* $[t_0 - \epsilon, t_0 + \epsilon]$*.*

4.1 Lipschitz Continuity of Activation Functions

For Picard's theorem to hold, the neural network's architecture $f(h,\theta)$ must be Lipschitz continuous. This depends entirely on the chosen activation function $\sigma(x)$. A function is Lipschitz continuous if there exists a constant $L > 0$ such that:

$$\|f(x) - f(y)\| \leq L\|x - y\| \quad \text{for all } x, y \tag{8}$$

Let us examine the two most common activation functions in deep learning:

1. **Hyperbolic Tangent (Tanh):** $\sigma(x) = \tanh(x)$. The derivative is $\sigma'(x) = 1 - \tanh^2(x)$. Because $0 \leq \sigma'(x) \leq 1$, the derivative is strictly bounded by 1. Therefore, $\tanh(x)$ is Lipschitz continuous with constant $L = 1$. A Neural ODE utilizing Tanh perfectly satisfies Picard's Theorem.

2. **Rectified Linear Unit (ReLU):** $\sigma(x) = \max(0, x)$. This function is non-differentiable at $x = 0$. However, it is easy to verify that $|\max(0,x) - \max(0,y)| \leq |x - y|$. Therefore, ReLU is uniformly Lipschitz continuous ($L = 1$). Picard's theorem guarantees

existence and uniqueness, proving continuous networks can safely utilize modern, non-smooth activations.

5 Training as a Boundary Value Problem

To train a continuous network, we must optimize the parameters θ to minimize a loss function $L(h(T))$, which evaluates the final state of the network. Because the network is defined by an ODE, we cannot use standard backpropagation. Instead, we must formulate a Boundary Value Problem (BVP) using the Adjoint Sensitivity Method.

5.1 The Adjoint State and Intuitive Derivation

We define the adjoint state $a(t)$ as the gradient of the loss with respect to the hidden state at any continuous depth t: $a(t) = \frac{\partial L}{\partial h(t)}$.

The question is how this gradient moves as t decreases. If we nudge $h(t)$ by a small amount, that perturbation gets carried forward by the dynamics f, and eventually shows up in the loss L. To track exactly how much the loss cares about the current state, we need to see how the Jacobian $\frac{\partial f}{\partial h}$ acts on the gradient as it travels backward. Working this out gives the adjoint ODE:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(h(t), \theta, t)}{\partial h} \tag{9}$$

The minus sign here is what makes this a backward equation — the gradient accumulates as t decreases, not increases, which is exactly what we want since we are integrating from $t = T$ back to $t = 0$..

5.2 Formulating the BVP

Training requires solving two coupled differential equations in opposite directions:

1. **Forward Pass (Initial Value Problem):** We know the input state at $t = 0$: $h(0) = x$. We integrate forward to $t = T$ to find the prediction.

2. **Backward Pass (Terminal Boundary Value Problem):** We evaluate the loss at $t = T$, which provides the terminal boundary condition: $a(T) = \frac{\partial L}{\partial h(T)}$. We then integrate the adjoint ODE backward from $t = T$ to $t = 0$ to compute the gradients.

6 Open Problem: Topological Constraints

While mathematically elegant, continuous neural networks possess strict theoretical limitations regarding their expressivity. This is a direct consequence of Picard's Uniqueness Theorem.

Because solutions to a Lipschitz-continuous ODE are unique, **trajectories in phase space cannot cross**. If two distinct input points ever occupied the exact same state h at the exact same depth t, they would be forced to follow the exact same path forever.

This means the network behaves like a homeomorphism — a continuous, smooth deformation of the input space. Points can be stretched or compressed, but the topology is preserved. The network cannot swap two points, break a loop, or disconnect a connected region..

6.1 The Expressivity Limitation

Consider a 1D dataset where we want the network to map $-1 \to 1$ and $1 \to -1$. For a continuous 1D ODE to achieve this, the trajectory starting at -1 must move right, and the trajectory starting at 1 must move left. Inevitably, these trajectories must intersect at some time t. However, Picard's theorem strictly forbids this intersection.

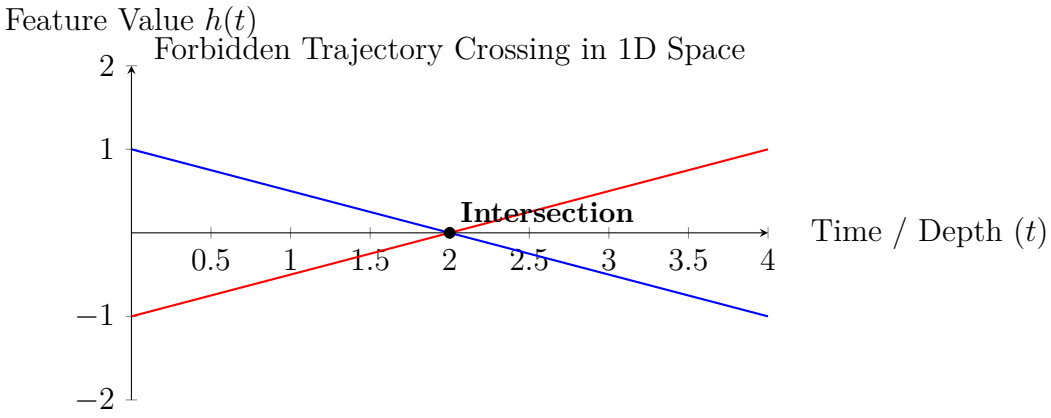

Figure 2: Visualizing the topological constraint. In a 1D continuous space, crossing trajectories violate uniqueness, meaning a standard Neural ODE cannot learn this simple mapping.

Therefore, a standard Neural ODE cannot learn this function. Current research focuses on *Augmented Neural ODEs*, which solve this by adding dummy dimensions to the ODE, allowing trajectories to lift into a higher-dimensional space and pass over each other without intersecting.

7 Computational Demonstration and Analysis

To validate this framework, we provide a computational comparison between a standard discrete ResNet step (Euler method) and a continuous Neural ODE forward pass.

```
import numpy as np
from scipy.integrate import solve_ivp

def f_dynamics(t, h, W, b):
    # The continuous derivative function dh/dt
```

```python
    return np.tanh(np.dot(W, h) + b)

# Model Parameters and Initial Condition
W = np.array([[-0.5, 0.8], [-1.2, 0.3]])
b = np.array([0.1, -0.2])
h0 = np.array([1.0, 0.5])

# 1. Discrete ResNet Step (Euler Method, dt=1.0)
h_euler = h0 + 1.0 * f_dynamics(0, h0, W, b)

# 2. Continuous Neural ODE (solve_ivp)
solution = solve_ivp(
    fun=lambda t, h: f_dynamics(t, h, W, b),
    t_span=(0.0, 1.0),
    y0=h0,
    method='RK45'
)
h_continuous = solution.y[:, -1]

print(f"Discrete (ResNet) Output:   {h_euler}")
print(f"Continuous (Neural ODE):    {h_continuous}")
```

Listing 1: Discrete vs. Continuous Forward Pass Integration

Analysis of Results:

```
Discrete (ResNet) Output:   [1. -0.34828364]
Continuous (Neural ODE):    [0.7213016  -0.35095462]
```

The numerical outputs reveal the effect of discretization when modeling neural network dynamics. The discrete Euler step (corresponding to a ResNet layer with step size $\Delta t = 1$) produces an approximation that deviates from the continuous Neural ODE solution. This deviation is more pronounced in the first component, while the second component remains relatively close.

To better understand this behavior, we visualize the trajectories of both methods over continuous depth.

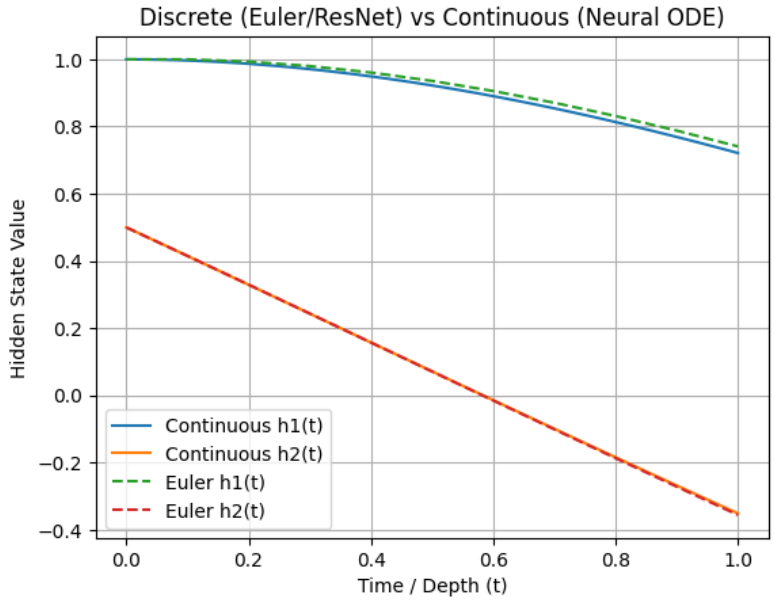

Figure 3: Comparison between discrete Euler (ResNet) approximation and continuous Neural ODE solution. The continuous trajectories are smooth, while the Euler approximation follows a stepwise path. The deviation between the two depends on the dynamics of each component.

From the plot, we observe that the Euler approximation closely follows the continuous trajectory in some components, while showing noticeable deviation in others. This indicates that the accuracy of the discrete approximation depends on the underlying dynamics of the system.

Thus, deep Residual Networks can be interpreted as coarse numerical approximations of continuous dynamical systems. The difference between discrete and continuous models arises from the choice of step size and numerical integration method, rather than an inherent limitation of the discrete architecture.

Furthermore, classical numerical analysis suggests that reducing the step size (i.e., increasing the number of layers) would improve the approximation, causing the discrete trajectory to converge toward the continuous solution.

8 Conclusion

The ResNet-to-ODE connection ends up being more than a curiosity. Once the forward pass is written as an IVP and training as a BVP, the whole framework of classical ODE analysis becomes available — and it actually says useful things. Picard-Lindelöf is not just a theoretical checkbox; it directly constrains which activation functions can be used and guarantees that the network behaves deterministically. The adjoint method is not just an alternative to backprop; it is the only coherent way to compute gradients when the "layers" are continuous. The topological constraint in Section 6 was, to me, the most unexpected result. It is a clean, mathematically precise statement about something that

is usually discussed vaguely — the idea that neural networks have limits. Here the limit is not about capacity or data, but about the geometry of ODE trajectories. A continuous 1D network simply cannot swap two points, and no amount of training will change that. Whether Augmented Neural ODEs or other extensions fully resolve these limitations remains an active area. What this project convinced me of is that differential equations are not just a metaphor for deep learning — they are the right language for asking precise questions about what these models can and cannot do.